Micro-Transformer: A From-Scratch Autograd and Attention Engine for Character-Level Language Modeling

Category: General Machine Learning

Team Members: Manikanta Harsha Nadimpalli, Luis Blanco Hoyos

Emails (SUNet): nmharsha@stanford.edu, blancol@stanford.edu

1 Motivation

Our primary motivation is pedagogical: we want to understand the architectural mechanics of modern language models by building the autograd machinery ourselves rather than treating it as a black box, and then use that machinery to study a concrete empirical question: how do LayerNorm placement and residual connections shape gradient magnitudes through attention during training? While PyTorch exposes hooks (`register_full_backward_hook`) and functional Jacobians (`torch.autograd.functional.jacobian`) that could surface much of this data, building forward and backward passes ourselves over batched tensors gives us both the conceptual clarity to motivate the ablation and exact, low-overhead access to intermediate Jacobian statistics without reliance on framework internals. The from-scratch build is the instrument; the gradient-flow ablation is the scientific contribution.

2 Methods

The framework is built in three layers. First, an autograd engine: a reverse-mode directed acyclic graph over tensor-valued nodes, extending Karpathy's scalar *micrograd* [2] to batched tensors with broadcasting. Second, a small set of linear-algebra primitives with explicit backward rules: batched matrix multiplication, broadcasting-aware reductions, and a numerically stable softmax. Third, a decoder-only transformer block built from these primitives, with explicit forward and backward passes for scaled dot-product attention [1], including closed-form Jacobians for the Q, K, and V projections. Normalization enters as a toggleable LayerNorm [3] layer whose placement (pre-norm, post-norm, or none) is the central variable in our ablation.

As a classical baseline drawn from CS229, we fit **softmax regression** (multi-class GLM) for next-character prediction using a one-hot context window over the same dataset and tokenizer. This anchors how much attention buys over a linear model studied directly in class.

3 Intended Experiments

We train character-level models on the Tiny Shakespeare corpus ($\sim$1.1MB of text, $\sim$65-character vocabulary), a standard small-scale benchmark for character-level language modeling that fits comfortably on a single GPU. We evaluate along two axes. First, **benchmarking**: we compare our custom framework against a functionally identical PyTorch transformer and the softmax-regression baseline on validation cross-entropy, held-out perplexity, steps to a perplexity threshold, peak memory, and wallclock per step. The softmax baseline anchors the lower bound; PyTorch anchors the upper bound on implementation efficiency.

Second, the primary experiment is a **gradient-flow ablation**: we cross LayerNorm placement (pre/post/none) with residual connections on/off and, for each configuration, log per-layer gradient norms, activation norms, and Jacobian singular-value statistics over training. These quantities our from-scratch autograd exposes directly but PyTorch abstracts away. Qualitative generation samples supplement the quantitative results.

4 Team Contributions

- **Manikanta Harsha Nadimpalli**: autograd engine (reverse-mode DAG over tensors), batched linear-algebra primitives, softmax-regression baseline, PyTorch benchmarking harness.
- **Luis Blanco Hoyos**: scaled dot-product attention (forward and backward), training loop, gradient-flow ablation instrumentation, and results analysis.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I. (2017). Attention is all you need. *NeurIPS 30*.

[2] Karpathy, A. (2020). micrograd: A tiny scalar-valued autograd engine. `https://github.com/karpathy/micrograd`.

[3] Ba, J.L., Kiros, J.R., Hinton, G.E. (2016). Layer Normalization. *arXiv:1607.06450*.